UTM UNIVERSITI TEKNOLOGI MALAYSIA | Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 1-2
Task : Project I
Due : 28th November 2025 (10%)

Name	Matric No.
GOE JIE YING	A23CS0224
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187
TEH RU QIAN	A23CS0191
WOO CHENG SHUAN	A23CS0283

Instructions

Each group must submit **one (1)** project only.

Your submission must include:

1. **Project Report** in **PDF format** (named as LeaderName_GroupNum_ProjectI.pdf).
2. The **demo video URL** (YouTube or Google Drive link) **inside the report**.
    - Make sure the link is **working**, **publicly accessible**, and does **not require login**.
3. Include all SQL script files (.sql) by attaching them directly, compressing them into one zip folder, or providing a valid OneDrive sharing link inside the report.

Presentation Video Link:
https://drive.google.com/drive/folders/1YJ7SeP9aJItN6g988id1XHyVNkcixe_C?usp=drive_link
GitHub Link: https://github.com/tanyiya/DP-Project/tree/main

Table of Contents

1.0 Introduction

The car rental business plays an important role in modern transportation by providing customers with flexible mobility alternatives without the long-term commitment of vehicle ownership. Travellers, professionals and those who need short-term access to cars frequently use rental services. Car rental companies are depending more and more on effective computerised systems to manage reservations, track vehicle availability, handle payments and store historical records as customer expectations increase and operations become more complex. A well-designed database system is thus required to ensure accuracy, reduce operational errors and enable quick data retrieval for decision-making.

By using a structured database backend, the Car Rental System aims to optimise core operating procedures. The main objective consist of:

1. **Efficient Data Storage**: Integrate all customer, vehicle, reservation, payment and employee data in one place.
2. **Accurate Data Management**: Ensure data integrity through proper relational design, normalization, constraints and indexing.
3. **Fast Data Retrieval**: Implement optimized SQL queries and indexing methods such as BTREE and FULLTEXT to improve search and retrieval performance.
4. **Operational Automation**: Reduce manual work by automating reservation validation, availability checking and billing computations.
5. **Data Security**: Protect sensitive customer and transaction data using encryption.

The Entity Relationship Diagram (ERD) shows the relationship of the car rental system. It includes four main entities: Vehicles, Customers, Bookings and Payments. Each vehicle can be booked by a customer and each booking may have one or more payments. The diagram shows how these entities interact with each other through foreign keys, ensuring data integrity and supporting efficient rental operations.

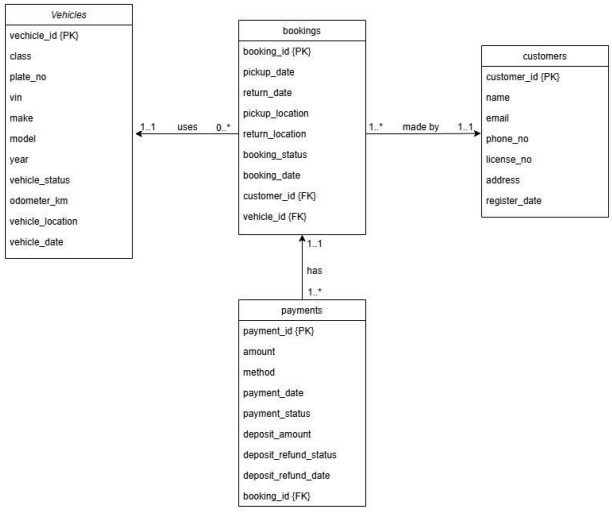


Figure 1.0 ERD of Car Rental System

The table shows the role of team members.

Team Member	Role and Responsibilities
Teh Ru Qian	**Database Designer** - Designs the database schema, creates tables with constraints and prepares the ERD
Tan Yi Ya	**SQL Implementer** - Performs data insertion, updates, deletions and summarizes SQL implementation
Goe Jie Ying	**Data Retrieval Analyst** - Implements SELECT queries, joins, filtering and shows data retrieval operations.
Woo Cheng Shuan	**Query Performance Engineer** - Implements BTREE indexes and analyzes query performance improvements
Lam Yoke Yu	**Text Data Optimization Engineer** - Implements TEXT indexing and contributes to performance optimization

2.0 SQL Implementation Summary

2.1 File Descriptions

The database structure, data manipulation and retrieval logic are distributed across these SQL files:

sql file	Descriptions
create_insert.sql	This file is responsible for **Database Definition Language (DDL)** and **Data Manipulation Language (DML)**. It contains the commands to create the car_rental database, define all tables, establish Foreign Key relationships using ALTER TABLE and populate the tables with initial sample data using INSERT statements.
update_delete.sql	This file contains essential **DML** statements for maintaining data integrity and currency. It provides examples of **UPDATE** statements to modify existing data, such as changing a booking status or a vehicle status and **DELETE** statements for conditional record removal, such as deleting payments and bookings linked to old, cancelled reservations.
dataRetrieval.sql	This file demonstrates various **Data Query Language (DQL)** techniques. It includes complex SELECT statements for filtering using WHERE, BETWEEN, LIKE, IS NULL, sorting using ORDER BY, LIMIT, aggregation using COUNT, SUM, AVG, MAX, MIN, grouping using GROUP BY, HAVING, string/numeric functions, conditional logic with CASE WHEN, subqueries, set operations using UNION, NOT EXISTS and different types of joins such as NATURAL JOIN, INNER JOIN, LEFT JOIN, SELF JOIN.
fulltextIndex.sql	This file focuses on performance optimization by creating a **FULLTEXT INDEX** on the pickup_location column of the bookings table. It includes EXPLAIN SELECT statements to illustrate the query plan difference and performance benefit of using MATCH AGAINST syntax over the LIKE operator for text searches.
btreeIndex.sql	This file demonstrates **BTree Indexing** to optimize standard data lookups on the bookings table. It creates an index on the pickup_date column (idx_pickup_date) and uses EXPLAIN SELECT to show how the database's query plan changes, resulting in faster retrieval for queries that filter by a specific date.

2.2 Constraints Used and Justification

Constraints are rules enforced on data columns to limit the type of data that can go into a table, ensuring data accuracy and reliability.

Constraint	Application Example	Purpose
PRIMARY KEY	vehicle_id in vehicles table	Uniquely identifies each record in a table and serves as the main lookup key.
FOREIGN KEY	customer_id and vehicle_id in bookings table	Enforces a link between two tables, ensuring referential integrity. Prevents creating a booking for a customer or vehicle that doesn't exist.
NOT NULL	class, make, model, year in vehicles	Ensures that a column must contain a value, preventing missing critical data fields.
UNIQUE	plate_no, vin in vehicles	Ensures all values in a column are different, preventing duplicate vehicle registration numbers or VINs.
CHECK	year in vehicles must be >= 1990	Defines a condition that each row must satisfy, ensuring vehicle year data is logically valid for a modern rental fleet.
DEFAULT	vehicle_status in vehicles is 'available'	Provides a default value for a column when none is specified, simplifying data entry for new vehicles.

2.3 Example SQL Snippets

2.3.1 CREATE TABLE (DDL)

This snippet demonstrates the table schema for **vehicles**, including the use of **PRIMARY KEY**, **UNIQUE**, **NOT NULL**, and **CHECK** constraints.

```SQL
CREATE TABLE vehicles (
     vehicle_id INT AUTO_INCREMENT PRIMARY KEY,
    class VARCHAR(20) NOT NULL,
```

```
    plate_no VARCHAR(20) UNIQUE NOT NULL,
    vin VARCHAR(17) UNIQUE NOT NULL,
    make VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    year INT NOT NULL CHECK (year >= 1990),
     vehicle_status VARCHAR(20) DEFAULT 'available',
    odometer_km INT DEFAULT 0,
    vehicle_location TEXT,
    vehicle_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE bookings
ADD customer_id INT NOT NULL,
ADD vehicle_id INT NOT NULL,
ADD FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
ADD FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id);
```

2.3.2 INSERT (DML)

An example of populating the **vehicles** table with new data.

SQL

```
INSERT INTO vehicles (class, plate_no, vin, make, model, year,
vehicle_status, odometer_km, vehicle_location)
VALUES
('Sedan', 'ABC123', '1HGCM82633A004352', 'Toyota', 'Vios', 2020,
'available', 25000, 'Kuala Lumpur');
```

2.3.3 SELECT

A **SELECT** query using an **INNER JOIN** to retrieve booking details along with the customer's name, demonstrating how data from multiple related tables is combined.

```sql
SELECT b.booking_id, c.name AS customer_name, b.pickup_date,
b.return_date
FROM bookings b
INNER JOIN customers c
    ON b.customer_id = c.customer_id;
```

2.3.4 UPDATE and DELETE (DML)

Examples of modifying and removing data. The **UPDATE** changes the booking_status for a specific booking.

```sql
UPDATE bookings
SET booking_status = 'completed'
WHERE booking_id = 10;
```

The **DELETE** uses a subquery to remove cancelled bookings before a specific date, ensuring data cleanup.

```sql
DELETE FROM bookings
WHERE booking_id IN (
    SELECT booking_id
    FROM (
        SELECT booking_id
        FROM bookings
        WHERE booking_status = 'cancelled'
          AND pickup_date < '2025-03-01'
    ) AS temp_bookings
);
```

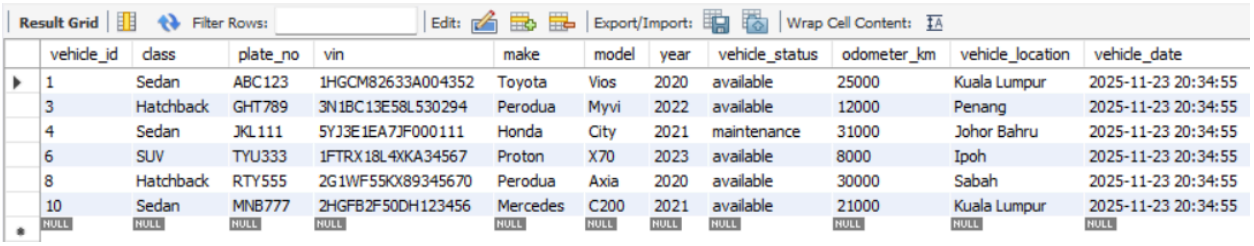
3.0 Query Demonstrations

3.1 Filtering

Code (BETWEEN & AND):

```
-- 1.2 Vehichle between 2020 and 2023 (BETWEEN, AND)
SELECT *
FROM vehicles
WHERE year BETWEEN 2020 AND 2023;
```

Result:



vehicle_id	class	plate_no	vin	make	model	year	vehicle_status	odometer_km	vehicle_location	vehicle_date
1	Sedan	ABC123	1HGCM82633A004352	Toyota	Vios	2020	available	25000	Kuala Lumpur	2025-11-23 20:34:55
3	Hatchback	GHT789	3N1BC13E58L530294	Perodua	Myvi	2022	available	12000	Penang	2025-11-23 20:34:55
4	Sedan	JKL111	5YJ3E1EA7JF000111	Honda	City	2021	maintenance	31000	Johor Bahru	2025-11-23 20:34:55
6	SUV	TYU333	1FTRX18L4XKA34567	Proton	X70	2023	available	8000	Ipoh	2025-11-23 20:34:55
8	Hatchback	RTY555	2G1WF55KX89345670	Perodua	Axia	2020	available	30000	Sabah	2025-11-23 20:34:55
10	Sedan	MNB777	2HGFB2F50DH123456	Mercedes	C200	2021	available	21000	Kuala Lumpur	2025-11-23 20:34:55
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

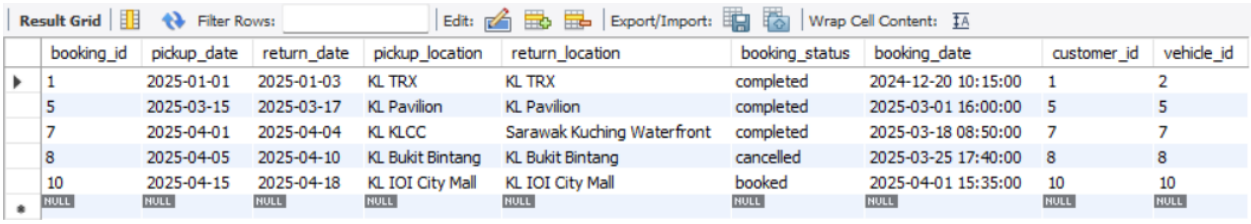
Explanation:

This query uses the BETWEEN operator together with AND to filter and retrieve vehicles manufactured between the years 2020 and 2023. It displays only vehicles that fall within this range in the result table.

Code (LIKE):

```
-- 1.5 Bookings that picked up in KL (LIKE)
SELECT *
FROM bookings
WHERE pickup_location LIKE '%KL%';
```

Result:



booking_id	pickup_date	return_date	pickup_location	return_location	booking_status	booking_date	customer_id	vehicle_id
1	2025-01-01	2025-01-03	KL TRX	KL TRX	completed	2024-12-20 10:15:00	1	2
5	2025-03-15	2025-03-17	KL Pavilion	KL Pavilion	completed	2025-03-01 16:00:00	5	5
7	2025-04-01	2025-04-04	KL KLCC	Sarawak Kuching Waterfront	completed	2025-03-18 08:50:00	7	7
8	2025-04-05	2025-04-10	KL Bukit Bintang	KL Bukit Bintang	cancelled	2025-03-25 17:40:00	8	8
10	2025-04-15	2025-04-18	KL IOI City Mall	KL IOI City Mall	booked	2025-04-01 15:35:00	10	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation:

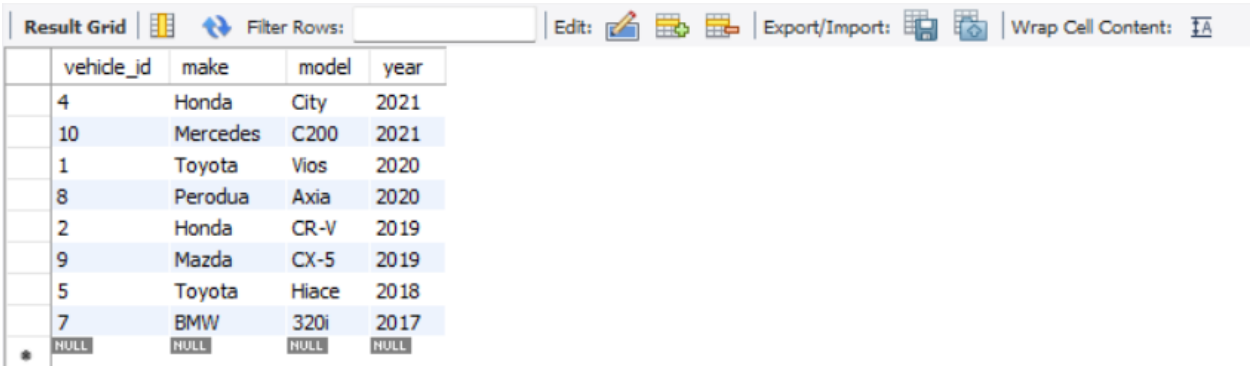
The query applies the LIKE '%KL%' condition to search for any pickup locations that include the characters “KL” anywhere in the text. All bookings with pickup locations in or containing “KL” are displayed in the result table.

3.2 Sorting

Code (ORDER BY):

```
-- 2.1 Sort the vehichle by year (ORDER BY)
SELECT vehicle_id, make, model, year
FROM vehicles
ORDER BY year DESC;
```

Result:



vehicle_id	make	model	year
4	Honda	City	2021
10	Mercedes	C200	2021
1	Toyota	Vios	2020
8	Perodua	Axia	2020
2	Honda	CR-V	2019
9	Mazda	CX-5	2019
5	Toyota	Hiace	2018
7	BMW	320i	2017
NULL	NULL	NULL	NULL

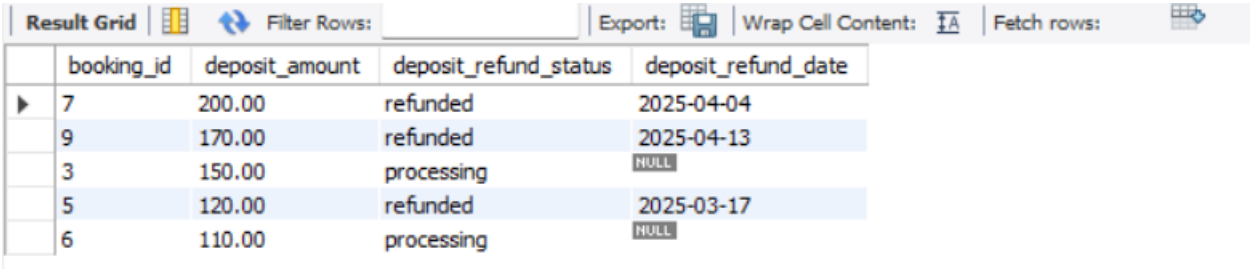
Explanation:

This query sorts the vehicles by their manufacturing year in descending order using the ORDER BY clause. This arranges the vehicles from the newest to the oldest in the result table.

Code (ORDER BY & LIMIT):

```
-- 2.2 Top 5 highest deposit amount (LIMIT)
SELECT booking_id, deposit_amount, deposit_refund_status, deposit_refund_date
FROM payments
ORDER BY deposit_amount DESC
LIMIT 5;
```

Result:



booking_id	deposit_amount	deposit_refund_status	deposit_refund_date
7	200.00	refunded	2025-04-04
9	170.00	refunded	2025-04-13
3	150.00	processing	NULL
5	120.00	refunded	2025-03-17
6	110.00	processing	NULL

Explanation:

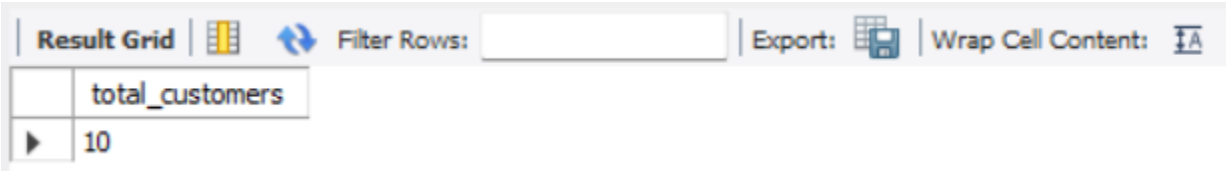
This query orders payment records by the deposit amount in descending order, then uses the LIMIT clause to display only the top five highest deposit amounts. It highlights the largest deposits from the payment list.

3.3 Aggregation

Code (COUNT):

```
-- 3.1 The number of customer (COUNT)
SELECT COUNT(*) AS total_customers
FROM customers;
```

Result:



total_customers
10

Explanation:

This query uses the COUNT function to calculate the total number of customers registered in the car rental system.

Code (MIN & MAX):

```
-- 3.4 Maximum and Minimum odometer reading (MAX & MIN)
SELECT
    MAX(odometer_km) AS highest_odometer,
    MIN(odometer_km) AS lowest_odometer
FROM vehicles;
```

Result:

highest_odometer	lowest_odometer
66000	8000

Explanation:

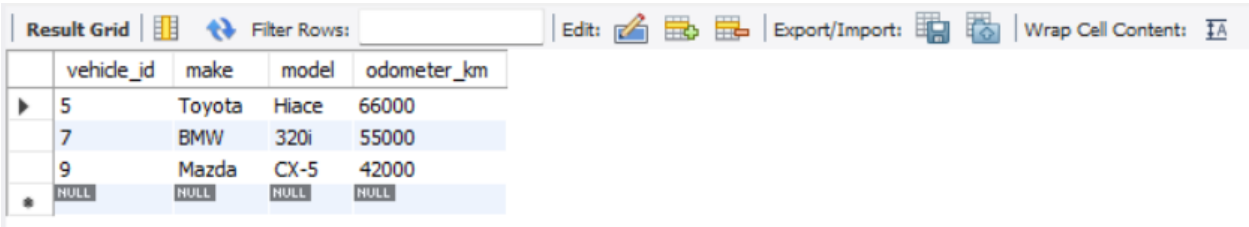
This query returns both the maximum and minimum odometer readings among all vehicles using the MAX and MIN functions. It helps the user to identify the range of mileage within the entire vehicle fleet.

3.4 Grouping and Filtering Groups

Code (GROUP BY & HAVING):

```
-- 4.2 Vechile with more that 40000km odometer reading (GROUP BY & HAVING)
SELECT vehicle_id, make, model, odometer_km
FROM vehicles
GROUP BY vehicle_id, make, model, odometer_km
HAVING odometer_km > 40000;
```

Result:



vehicle_id	make	model	odometer_km
5	Toyota	Hiace	66000
7	BMW	320i	55000
9	Mazda	CX-5	42000
NULL	NULL	NULL	NULL

Explanation:

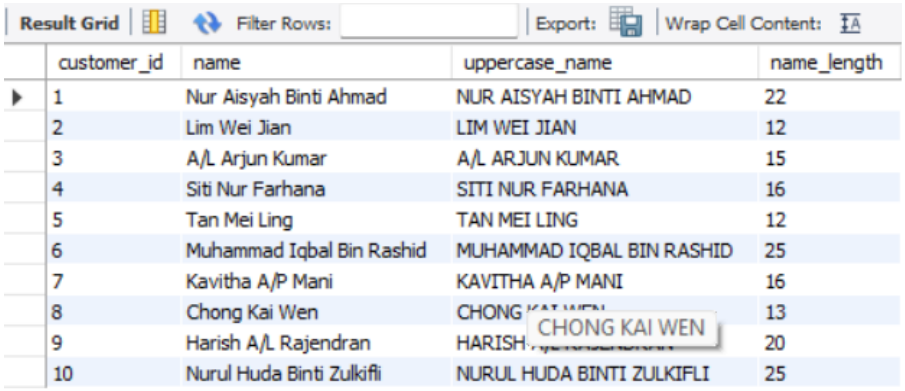
This query uses GROUP BY to group vehicle records based on their ID, make, model, and odometer reading. The HAVING clause is then applied to filter the grouped results, showing only vehicles with an odometer reading greater than 40,000 km. This demonstrates how HAVING is used to filter data after grouping.

3.5 Numeric and String Functions

Code (UPPER & LENGTH):

```
-- 5.2 Change Customer Name to UPPERCASE and count the length (UPPER & LENGTH)
SELECT customer_id, name, UPPER(name) AS uppercase_name, LENGTH(name) AS name_length
FROM customers;
```

Result:



customer_id	name	uppercase_name	name_length
1	Nur Aisyah Binti Ahmad	NUR AISYAH BINTI AHMAD	22
2	Lim Wei Jian	LIM WEI JIAN	12
3	A/L Arjun Kumar	A/L ARJUN KUMAR	15
4	Siti Nur Farhana	SITI NUR FARHANA	16
5	Tan Mei Ling	TAN MEI LING	12
6	Muhammad Iqbal Bin Rashid	MUHAMMAD IQBAL BIN RASHID	25
7	Kavitha A/P Mani	KAVITHA A/P MANI	16
8	Chong Kai Wen	CHONG KAI WEN	13
9	Harish A/L Rajendran	HARISH	20
10	Nurul Huda Binti Zulkifli	NURUL HUDA BINTI ZULKIFLI	25

Explanation:

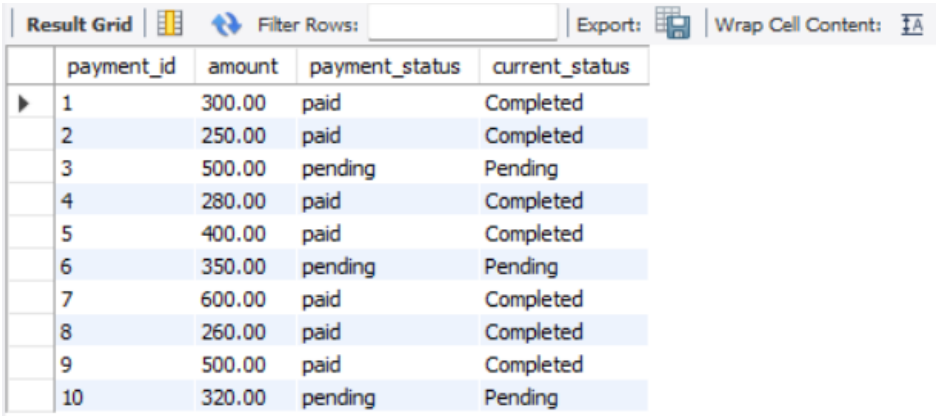
This query applies the UPPER() function to convert each customer's name into uppercase letters and uses the LENGTH() function to calculate the number of characters in the name.

3.6 Conditional Logic

Code (CASE WHEN):

```
-- 6.1 Categorize payments (CASE WHEN)
SELECT payment_id, amount, payment_status,
       CASE
           WHEN payment_status = 'paid' THEN 'Completed'
           WHEN payment_status = 'pending' THEN 'Pending'
           ELSE 'Unknown'
       END AS current_status
FROM payments;
```

Result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_id	amount	payment_status	current_status
1	300.00	paid	Completed
2	250.00	paid	Completed
3	500.00	pending	Pending
4	280.00	paid	Completed
5	400.00	paid	Completed
6	350.00	pending	Pending
7	600.00	paid	Completed
8	260.00	paid	Completed
9	500.00	paid	Completed
10	320.00	pending	Pending

Explanation:

This query uses the CASE WHEN expression to classify each payment based on its status. If the payment status is 'paid', it is labeled as “Completed”. If the status is 'pending', it is marked as “Pending”. Any other status is categorized as “Unknown”. The label represents the current status of each payment.

3.7 Subqueries

Code (Multiple-row):

```
-- 7.2 Customer that booked a 'SUV' vehicle (Multi-row)
SELECT *
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM bookings
    WHERE vehicle_id IN (
        SELECT vehicle_id
        FROM vehicles
        WHERE class = 'SUV'
    )
);
```

Result:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone_no	license_no	address	register_date
1	Nur Aisyah Binti Ahmad	aisyah.ahmad@example.com	0123456789	L1234567	Selangor	2025-11-23 20:35:01
6	Muhammad Iqbal Bin Rashid	iqbal.rashid@example.com	0178901234	L6666777	Perak	2025-11-23 20:35:01
9	Harish A/L Rajendran	harish.raj@example.com	0112345678	L3333444	Sarawak	2025-11-23 20:35:01
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation:

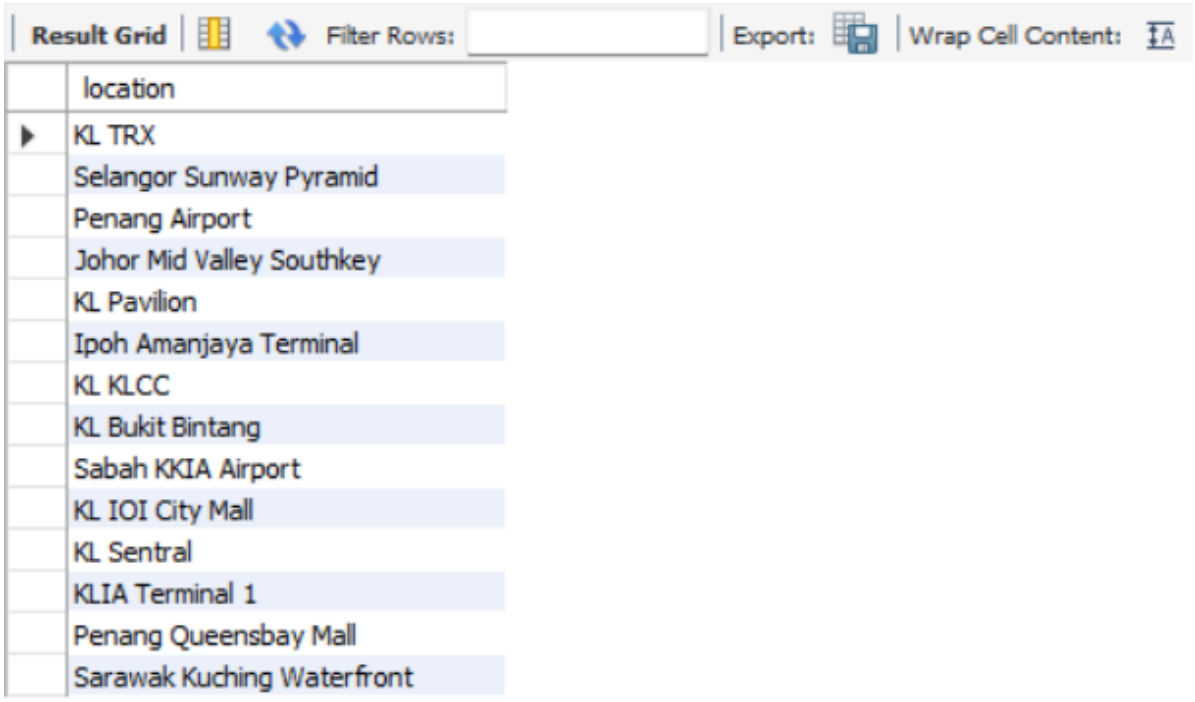
This query uses a multi-row subquery to find all customers who have booked a vehicle of class ‘SUV’. The inner subquery retrieves vehicle IDs of SUVs, and the outer query matches these to customer bookings. It shows how subqueries can return multiple values to filter records in the main query.

3.8 Set operations

Code (UNION):

```
-- 8.1 List all pickup and return locations (UNION)
SELECT pickup_location AS location
FROM bookings
UNION
SELECT return_location
FROM bookings;
```

Result:



location
KL TRX
Selangor Sunway Pyramid
Penang Airport
Johor Mid Valley Southkey
KL Pavilion
Ipoh Amanjaya Terminal
KL KLCC
KL Bukit Bintang
Sabah KKIA Airport
KL IOI City Mall
KL Sentral
KLIA Terminal 1
Penang Queensbay Mall
Sarawak Kuching Waterfront

Explanation:

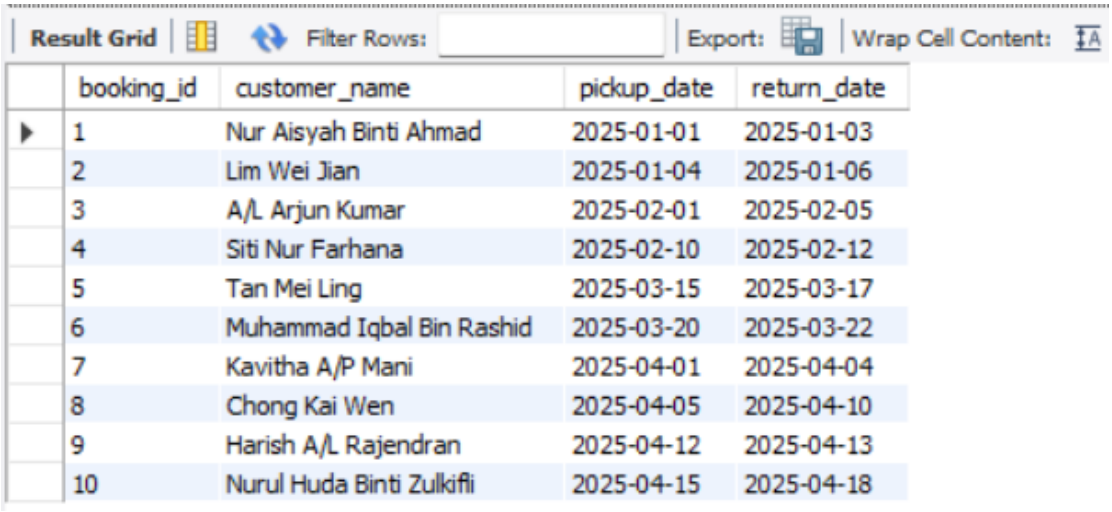
This query uses the UNION operator to combine all pickup locations and all return locations from the bookings table. It automatically removes duplicate values to form an unique list, so each location appears only once in the result table.

3.9 Joins

Code (Inner Join):

```
-- 9.2 Booking with customer's name (INNER)
SELECT b.booking_id, c.name AS customer_name, b.pickup_date, b.return_date
FROM bookings b
INNER JOIN customers c
    ON b.customer_id = c.customer_id;
```

Result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

booking_id	customer_name	pickup_date	return_date
1	Nur Aisyah Binti Ahmad	2025-01-01	2025-01-03
2	Lim Wei Jian	2025-01-04	2025-01-06
3	A/L Arjun Kumar	2025-02-01	2025-02-05
4	Siti Nur Farhana	2025-02-10	2025-02-12
5	Tan Mei Ling	2025-03-15	2025-03-17
6	Muhammad Iqbal Bin Rashid	2025-03-20	2025-03-22
7	Kavitha A/P Mani	2025-04-01	2025-04-04
8	Chong Kai Wen	2025-04-05	2025-04-10
9	Harish A/L Rajendran	2025-04-12	2025-04-13
10	Nurul Huda Binti Zulkifli	2025-04-15	2025-04-18

Explanation:

This query uses an INNER JOIN to combine the bookings and customers tables based on matching customer_id. It retrieves each booking along with the corresponding customer's name and booking dates. Only bookings that have a matching customer record are included in the result table.

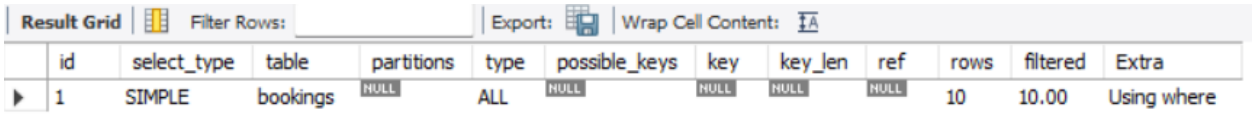
4.0 Optimisation Section

4.1 BTREE Index

Before indexing:

```
EXPLAIN SELECT * FROM bookings
WHERE pickup_date = '2025-03-15';
```

Select result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	bookings	NULL	ALL	NULL	NULL	NULL	NULL	10	10.00	Using where

It scans every row in the bookings table as part of a full table scan from 'ALL' type mentioned. Since pickup_date does not have any index, it must compare the date value row by row. This is ineffective and slows down as the table gets bigger.

Indexing:

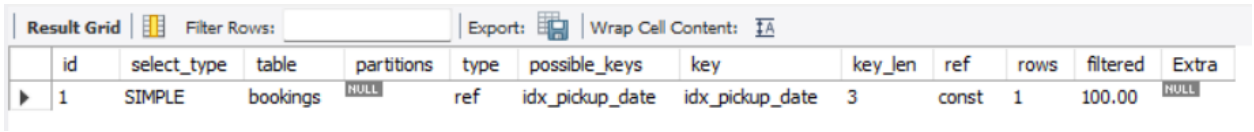
```
CREATE INDEX idx_pickup_date
ON bookings(pickup_date);
```

A BTREE index is created on the pickup_date column. Because BTREE enables it to quickly find matched values via tree traversal rather than scanning the full table, it is perfect for date, numeric and range-based queries.

After indexing:

```
EXPLAIN SELECT * FROM bookings
WHERE pickup_date = '2025-03-15';
```

Select result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	bookings	NULL	ref	idx_pickup_date	idx_pickup_date	3	const	1	100.00	NULL

It uses the index directly to find matched rows, which is an improvement over ALL -> ref. The fact that idx_pickup_date is now recognized and actively used by possible_keys and key. Rows examined decrease from 10 rows to 1 row after indexing.

	Before Indexing	**After Indexing**
Type	ALL	ref

Key	NULL	idx_pickup_date
Possible Keys	NULL	idx_pickup_date
Rows	10	1
Methods	Full table scan	Index lookup

4.2 FULLTEXT Index

Before indexing:

```sql
EXPLAIN SELECT * FROM bookings
WHERE pickup_location LIKE '%KL%';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	bookings	NULL	ALL	NULL	NULL	NULL	NULL	10	11.11	Using where

Select Results:

booking_id	pickup_date	return_date	pickup_location	return_location	booking_status	booking_date	customer_id	vehicle_id
1	2025-01-01	2025-01-03	KL TRX	KL TRX	completed	2024-12-20 10:15:00	1	2
5	2025-03-15	2025-03-17	KL Pavilion	KL Pavilion	completed	2025-03-01 16:00:00	5	5
7	2025-04-01	2025-04-04	KL KLCC	Sarawak Kuching Waterfront	completed	2025-03-18 08:50:00	7	7
8	2025-04-05	2025-04-10	KL Bukit Bintang	KL Bukit Bintang	cancelled	2025-03-25 17:40:00	8	8
10	2025-04-15	2025-04-18	KL IOI City Mall	KL IOI City Mall	booked	2025-04-01 15:35:00	10	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Indexing:

```sql
CREATE FULLTEXT INDEX ft_pickup_location ON bookings(pickup_location);
```

Indexes in Bookings

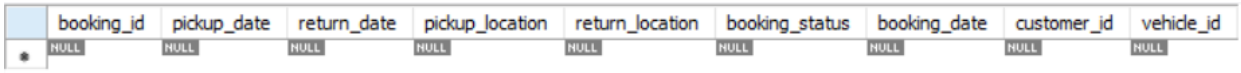
Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
bookings	0	PRIMARY	1	booking_id	A	10	NULL	NULL		BTREE			YES	NULL
bookings	1	customer_id	1	customer_id	A	10	NULL	NULL		BTREE			YES	NULL
bookings	1	vehicle_id	1	vehicle_id	A	10	NULL	NULL		BTREE			YES	NULL
bookings	1	ft_pickup_location	1	pickup_location	NULL	10	NULL	NULL	YES	FULLTEXT			YES	NULL

After indexing:

```sql
EXPLAIN SELECT * FROM bookings
WHERE MATCH(pickup_location) AGAINST('KL' IN NATURAL LANGUAGE MODE);
```

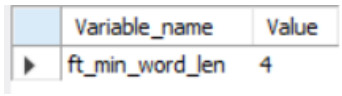
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	bookings	NULL	fulltext	ft_pickup_location	ft_pickup_location	0	const	1	100.00	Using where; Ft_hints: sorted

Select Results:



booking_id	pickup_date	return_date	pickup_location	return_location	booking_status	booking_date	customer_id	vehicle_id
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

The results show `NULL` due to the minimum word length restriction for FULLTEXT indexing, which is set to 4, as shown below:



Variable_name	Value
ft_min_word_len	4

This is understandable, as the minimum word length of 4 is intended to prevent indexing short words that don't add significant meaning on their own (e.g., "to", "is", "and"). Indexing such short words would create many unnecessary entries, which could increase processing time and reduce query performance.

When we change the search text to text more than 4 characters, the `SELECT` statement works perfectly fine.

However, when comparing the execution plan using the `EXPLAIN` keyword, the results are as follows:

	Before Indexing	After Indexing
SELECT type	`ALL`	`fulltext`
Rows examined	10 (All)	1

As we can see, the `SELECT` type is different, and there is a significant decrease in the number of rows examined.

Full text index can be also implemented on `bookings(return_location)`.

5.0 Team Work

Team Member	Role and Responsibilities
Teh Ru Qian	I was responsible for designing the entire database structure. This included planning out the schema, creating all the tables and setting up the primary keys, foreign keys and constraints to make sure the data stayed consistent. I also created the ERD to clearly show how all the tables connect and support the system. I improved my understanding of how different tables relate to one another in a real system. Designing everything from scratch and ensuring that each connection worked properly helped me appreciate how important good relational design is. Overall, the experience taught me how a carefully planned schema can keep a system organised, efficient and easy to maintain.
Tan Yi Ya	My role included the insertion of all sample data into the car_rental database tables, ensuring the system was fully populated and ready for testing. I was also responsible for implementing data maintenance operations, which is writing the UPDATE and DELETE queries to manage record modifications and conditional cleanup. Furthermore, I am in charge of the SQL Implementation Summary part for the project report, detailing the function of each SQL file, listing key code snippets and explaining the use of constraints. I learned a lot about the relational logic between the tables. By working on data insertion and then writing UPDATE and DELETE queries that relied on navigating Foreign Key relationships, I gained a solid understanding of relational database principles and how to maintain data integrity across the entire schema.
Goe Jie Ying	I was responsible for retrieving data using SELECT statements to demonstrate different situations such as filtering, aggregation, and subqueries. By applying various SQL clauses, I was able to extract meaningful and relevant information, which is essential for supporting decision-making. My role also included demonstrating queries in the report, as well as presenting and explaining the results for each output. Through this project, I gained experience with a wide range of SQL keywords and techniques, including LIKE, CASE WHEN, and different types of subqueries, such as multiple-row and correlated subqueries, which I had not used before. This knowledge will definitely help me deepen my understanding and explore more advanced concepts in database management.

Woo Cheng Shuan	I was in charge of analyzing and enhancing SQL query performance in the Car Rental System database. Creating and testing BTREE indexed on commonly searched date and numeric columns, including pickup_date, accessing query plans using the EXPLAIN statement and comparing query performance before and after indexing were my responsibilities. Also, I made sure that the database searches function well as the volume of data increases, explained changes in type, possible key and rows evaluated and recorded the optimization findings. I was able to learn more about how MySQL's query optimiser works, how indexing methods affect performance, and how to interpret execution plans to solve real database performance problems. Through this task, I learned how indexing helps reduce unnecessary scans and improves overall performance as the database grows.
Lam Yoke Yu	I was responsible for optimising the database, particularly for text searches. A FULLTEXT index was created on the pickup_location and return_location fields in the bookings table. To evaluate performance improvements, I used the EXPLAIN statement. In this project, I learned how a FULLTEXT index can significantly improve query performance, especially as the database continues to grow in size. I also encountered challenges during implementation, such as the minimum word length required for FULLTEXT indexing. Through this, I learned that adjusting the configuration allows us to customize indexing to fit our requirements.

6.0 Conclusion

During the development of the **car_rental** database, several challenges were encountered that contributed to a deeper understanding of relational design and SQL implementation. One major challenge involved **maintaining referential integrity across tables** that depend on each other. Designing Foreign Key relationships that follow real-world rules while still allowing updates or deletions required careful planning. Therefore, we spent some time discussing the logic during database creation before moving on to the next stage.

Another challenge appeared during the creation of advanced queries, which is **combining multiple joint subqueries and filtering conditions**. It was necessary to understand how different join types behave in various scenarios, and required multiple tests to ensure accuracy. The use of indexing also added another layer of complexity because it required awareness of storage engine requirements and differences between natural-language search and pattern matching.

There are several ways to further improve the system. Adding **more business logic constraints** would make the database more reliable. For example, rules that prevent overlapping vehicle bookings or enforce deposit conditions could be added to strengthen data integrity. Without a frontend, the use of **stored procedures triggers** or views can also help automate common tasks and improve maintainability. Performance can be enhanced through the use of **composite indexes** on columns that are frequently used in joins such as customer_id or vehicle_id.

Overall the project achieved its goals and provided valuable experience in solving challenges related to database structure query design and performance optimization. Further refinement in these areas would help the system become more scalable, more efficient and more suitable for real-world use.

Marking Rubric

SQL Implementation (CLO1)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Backend SQL Implementation (DDL/DML/Queries)	All SQL statements execute flawlessly; covers full range – DDL, DML, SELECT, joins, subqueries, functions.	Most SQL correct; small syntax or logic issues.	Several working statements but limited variety.	Few working statements; repetitive or basic logic.	Non-functional / incomplete SQL.	
Advanced Data Logic (Filtering, Aggregation, Subquery)	Uses complex logic (nested queries, GROUP BY, HAVING, CASE, conditions) effectively.	Correct use of aggregations with minor gaps.	Some queries are too simple or misapplied.	Few aggregations or logical errors.	Not attempted.	
Indexing & Optimization	Correctly creates BTREE and TEXT indexes; shows analysis before/after; interprets performance gain.	Indexes implemented; partial analysis.	Indexing attempted without analysis.	Incomplete index usage.	Not attempted.	
Transaction Control & Front-End Logic Integration	Demonstrates atomic operations (START TRANSACTION, COMMIT, ROLLBACK); explains how it fits system workflow (front-end use case e.g. booking/purchase).	Transaction logic corrects but limited integration explanation.	Partial transaction implementation.	Minimal attempt, unclear scenario.	Missing transaction logic.	
Subtotal						

Teamwork and Collaboration (CLO3)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Team Coordination & Contribution	Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work.	Good teamwork and participation with minor imbalance.	Some collaboration; limited task clarity.	Minimal teamwork; dominated by few members.	No teamwork evidence; incomplete group submission.	
Subtotal						

Documentation & Presentation (CLO4)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Report Documentation	Clear and complete report including intro, SQL summary, query screenshots, optimization, transaction explanation, and reflection.	Report complete with minor formatting or clarity issues.	Adequate report but lacks details or screenshots.	Weakly structured or incomplete sections.	Missing or very poor documentation.	
Slide Quality & Visual Communication	Slides are professional, organised, visually clear, and consistent with project flow.	Well-prepared slides with minor design issues.	Slides understandable but inconsistent layout.	Poorly structured or cluttered slides.	Missing or unreadable slides.	
Verbal Presentation	Excellent delivery, confident speakers, balanced time usage, fluent explanation, and effective Q&A handling.	Good delivery; some coordination issues.	Understandable but low energy; moderate Q&A.	Disorganized delivery or weak explanations.	Unable to present or answer questions.	
Subtotal						